

Capgraph: Explicit Capability Relations for Agent Skills - Design and Controlled Evaluation

Tino Wening\Independent Researcher

Technical Report - Version 1.0, August 2026

Contents

Abstract	2
1. Introduction	2
2. Background and Related Work	3
2.1 Agent Skills and progressive disclosure	3
2.2 Structured skill organization and retrieval	3
2.3 Executable skill graphs, verification, and recovery	4
2.4 Skill evolution and coding agents	4
2.5 Positioning of Capgraph	4
3. Capgraph Design	5
3.1 Design goals	5
3.2 Skill representation	5
3.3 External graph index	5
3.4 Relation semantics	6
3.5 Batch disclosure	6
4. Experimental Method	6
4.1 Research question	6
4.2 Agent and environment	7
4.3 Flat and Graph conditions	7
4.4 Independent verification	7
4.5 Metrics	7
5. Experiments	8
5.1 V0: shallow workflow and progressive graph loading	8
5.2 V0.1: loading-policy ablation	8
5.3 V1: larger composition and controlled recovery	8
6. Results	9
6.1 Reliability	9
6.2 Composition and irrelevant skill exposure	9
6.3 Capability context	9
6.4 Interaction and provider usage	10
6.5 Recovery	10
6.6 Shared dependency handling	10
7. Discussion	10
7.1 Structured composition can help without improving success	10
7.2 Loading policy is part of the method	11
7.3 Capability dependency is not necessarily context dependency	11
7.4 Flat selection has a genuine adaptive advantage	11
7.5 Batch loading is a benchmark confound as well as a design result	11
7.6 Known-root composition is intentionally not retrieval	12
7.7 Interoperability and metadata	12
8. Threats to Validity and Limitations	12
9. Reproducibility	13
10. Conclusion	13

AI Assistance Disclosure	13
References	13

Abstract

Agent Skills package procedural knowledge and supporting resources into reusable, on-demand context for language-model agents. The current Agent Skills format provides a lightweight discovery and activation mechanism, but skill-to-skill composition remains largely implicit: an agent may need to infer which skills are prerequisites, which capability should verify an outcome, and which recovery procedure is relevant after failure. This report presents **Capgraph**, a minimal capability-composition layer that encodes explicit `requires`, `verify_with`, and `recover_with` relations in standards-compatible Agent Skills metadata, indexes the resulting graph outside the model context, expands a known root capability deterministically, batch-loads the relevant execution and verification closure, and keeps recovery prose lazy until a relevant failure occurs.

Capgraph is evaluated as a controlled systems experiment rather than as a claim that skill graphs are novel. The evaluation comprises three benchmark stages and 36 formal model runs using the same coding-agent harness and independently verified UPBGE editor state. In the first stage, progressive one-skill-at-a-time graph loading added overhead without improving reliability. A loading-policy ablation then showed that deterministic batch loading removed most of this interaction overhead. In the final 24-capability benchmark, Flat and Graph conditions both achieved 4/4 independently verified success. Graph composition was more deterministic, loaded no irrelevant skill bodies, and exposed 43.6% less median measured capability context in the tested harness; it also reduced several interaction and provider-usage metrics, while median provider input tokens were 1.0% higher. Flat, however, sometimes omitted declared low-level skill prose safely.

The results support a narrow conclusion: explicit capability relations can reduce irrelevant capability exposure and make known-root composition more deterministic, but they did not improve final task success in the tested workloads. The experiments further motivate a distinction between **capability dependencies** and **model-context dependencies**, and show that loading policy is a first-order design variable when evaluating structured agent skills.

1. Introduction

Agent Skills have emerged as a lightweight mechanism for giving language-model agents reusable procedural knowledge without retraining. A skill is centered on a `SKILL.md` file containing YAML metadata and Markdown instructions, optionally accompanied by scripts, references, and assets [1]. The format is intentionally simple. In the common progressive-disclosure pattern, compact metadata is available during discovery and the full skill body is loaded only when the agent activates that skill [1].

This design makes individual skills portable, but it leaves a composition problem. Real tasks often require more than one procedural capability. A high-level operation may depend on lower-level setup steps; successful execution may need a specific verifier; and a structured failure may imply a particular recovery procedure. In a flat skill catalog, these relations are not first-class. The model may infer them from names, descriptions, prose, tool observations, or previous experience. That flexibility is useful, but it can also cause unnecessary exploration, redundant context loading, or inconsistent composition as the catalog grows.

This problem is already recognized in both research and the Agent Skills community. Proposals in the Agent Skills repository have discussed prerequisite and related-skill fields as well as package-level dependency manifests [2, 3]. Recent research systems organize skills with trees, directed acyclic graphs, typed skill graphs, dependency-aware retrieval, execution verification, repair, and learned or evolving skill relations [4–10]. Therefore, Capgraph does **not** claim to introduce the idea of a skill graph, skill dependencies, graph-based verification, or skill relations in frontmatter.

Instead, this work asks a narrower systems question:

Given a known root capability, does an explicit capability graph help an agent compose, verify, and recover skills more reliably or efficiently than normal flat skill discovery?

Capgraph deliberately isolates that question from natural-language skill routing. The root capability is supplied directly in every formal task. There is no semantic retrieval layer, embedding index, vector database, learned router, automatic graph evolution, or multi-agent planner. The purpose is to measure what explicit relations themselves contribute once the entry point is known.

Capgraph makes four practical contributions:

1. **A minimal Agent-Skills-compatible reference design.** Relations are encoded through the existing extensible metadata field rather than by defining a new skill file format. The graph is indexed outside model context, while skill prose remains ordinary SKILL.md content.
2. **A deterministic known-root composition mechanism.** requires edges are recursively expanded and deduplicated; verification and recovery relations remain terminal; normal execution and verification prose can be batch-loaded; recovery prose is withheld until a relevant failure.
3. **A controlled loading-policy ablation.** The experiments show that one-skill-at-a-time graph loading can erase the expected context benefit through additional agent/tool interaction, while batch disclosure materially reduces this overhead.
4. **Mechanism-level empirical evidence rather than a reliability claim.** Across 36 formal runs, Flat and Graph conditions always reached independently verified success. In the larger benchmark, Graph reduced irrelevant capability exposure and made composition deterministic, while Flat retained an adaptive ability to omit some declared low-level prose.

The final observation is particularly important. A capability may be structurally required for successful execution without the language model necessarily needing to read that capability's full procedural prose. This suggests that **execution dependency and context dependency are related but not identical concepts**.

2. Background and Related Work

2.1 Agent Skills and progressive disclosure

The Agent Skills specification defines a skill as a directory containing a mandatory SKILL.md with YAML frontmatter and Markdown instructions [1]. Required metadata consists of name and description; optional fields include license, compatibility, metadata, and experimental allowed-tools. The specification permits arbitrary string key-value pairs in metadata, providing an extension point for client-specific properties [1].

The standard loading model is intentionally progressive. Skill names and descriptions provide a lightweight discovery surface, while full instructions and auxiliary resources are loaded when needed [1]. This reduces the need to place every skill body into the initial context window, but it does not itself define runtime relations between skills.

The absence of first-class relations has been discussed directly in the Agent Skills community. Issue #90 proposed `prerequisite-skills` and `related-skills` frontmatter fields so agents could reason about execution order and complementary skills without inferring every relationship from context [2]. Issue #100 separately asked how one skill should depend on or reuse another. A later package-manifest proposal moved installation and distribution dependencies into a separate `skills.json`, explicitly distinguishing distribution concerns from runtime composition [3].

Practical projects have also explored skill graphs. Skill Writer, for example, includes a `graph:` frontmatter block with typed relationships such as `depends_on`, graph health checks, and bundle resolution [11]. This is important prior art: relations stored near skill definitions, topological dependency handling, and graph-based bundles all predate Capgraph.

Capgraph therefore takes a deliberately narrower position. It uses the existing Agent Skills metadata extension mechanism rather than a new top-level relation schema, and studies runtime composition from a supplied root rather than package installation, skill discovery, or graph evolution.

2.2 Structured skill organization and retrieval

AgentSkillOS organizes large skill ecosystems using a capability tree for discovery and DAG-based pipelines for orchestration [4]. Its experiments span skill ecosystems from hundreds to hundreds

of thousands of skills and report advantages over flat invocation. This establishes that hierarchical organization and structured orchestration can be valuable at ecosystem scale.

Graph of Skills (GoS) constructs an offline executable skill graph and performs hybrid semantic-lexical seeding, graph-based propagation, and context-budgeted hydration to retrieve dependency-aware skill bundles from large libraries [5]. GoS directly targets a problem Capgraph leaves out of scope: mapping a task into the relevant region of a large skill space. Capgraph starts after this step by assuming a known root.

SkillGraph represents reusable skills as nodes with typed prerequisite, enhancement, and co-occurrence relations and evolves the graph using trajectories and reinforcement-learning feedback [7]. **SkillDAG** similarly treats inter-skill structure as an inference-time, agent-callable typed graph and supports execution-backed graph edits across episodes [8]. Both systems integrate graph structure with retrieval and learning/evolution, whereas Capgraph intentionally keeps the graph static during the experiment.

More recently, **SkillTrace** combines query decomposition, query-to-skill matching, and propagation over skill dependencies [10]. This again highlights the coupling between natural-language routing and structural composition. Capgraph isolates the latter so that routing quality does not dominate the experiment.

2.3 Executable skill graphs, verification, and recovery

GraSP introduces an executable skill-graph architecture that compiles flat skills into typed DAGs with precondition-effect edges, performs node-level verification, and uses locality-bounded repair [6]. **HiSkill** connects high-level skills to AtomicOps and models decomposition, temporal transition, compatibility, support, and recovery relations in a hierarchical skill graph [9]. These systems are substantially richer execution architectures than Capgraph.

Consequently, Capgraph does not claim novelty for graph-based verification or recovery. Its contribution is instead to test whether a much smaller relation model—`requires`, `verify_with`, and `recover_with`—can be layered onto ordinary Agent Skills and provide measurable composition benefits in an existing coding-agent harness.

2.4 Skill evolution and coding agents

Skill relations are also appearing in work on autonomous skill evolution. **GSE** maintains a Skill Relation Graph while learning globally reusable skills for software-engineering agents, combining relationship modeling with cluster-based consolidation and replay-driven verification [12]. This work targets long-term skill learning and generalization; Capgraph does not learn or modify skills or graph edges.

Empirical research is simultaneously beginning to treat `SKILL.md` as a software artifact in its own right. Hong et al. analyze real-world skills, authoring practices, and “skill smells,” reinforcing the view that skill structure and maintenance are engineering concerns rather than merely prompt-writing details [13].

2.5 Positioning of Capgraph

The closest prior work already covers most broad graph claims. Skill dependencies, typed relations, topological composition, verification, recovery, graph-based retrieval, and evolving skill graphs are all established directions [4–12]. The purpose of Capgraph is therefore not to compete on architectural breadth.

Its distinct experimental focus is the following combination:

- existing Agent Skills remain the unit of procedural knowledge;
- explicit relations are encoded through the standard `metadata` extension surface;
- the complete graph stays outside the language-model context;
- the task supplies a known root capability;
- `requires` defines a deterministic transitive execution closure;
- verification is selected structurally;

- recovery prose is lazy;
- loading policy is explicitly ablated;
- Flat and Graph conditions are compared inside the same executable coding-agent harness with independent environment verification.

The emphasis is thus on **reference design and controlled systems evaluation**, not on claiming the first skill graph.

3. Capgraph Design

3.1 Design goals

Capgraph was developed with five constraints.

First, it should not introduce a parallel proprietary skill format. Existing SKILL.md bodies should remain usable by normal Agent-Skills-compatible tooling.

Second, graph structure should not itself consume substantial model context. The complete registry and graph should remain harness state.

Third, deterministic work should stay outside the model. Dependency traversal, cycle detection, reference validation, and deduplication are graph operations rather than language-model reasoning tasks.

Fourth, execution, verification, and recovery should be distinguishable. A verifier is not an execution prerequisite, and recovery prose should not be eagerly loaded during a successful normal path.

Fifth, the implementation should remain deliberately small enough to falsify the core hypothesis before adding retrieval, learning, persistence, or orchestration infrastructure.

3.2 Skill representation

A Capgraph-managed skill remains an Agent Skills directory with a conventional SKILL.md. The skill body contains operational guidance. Additional graph relations are encoded as namespaced string metadata.

Conceptually:

```

---
name: vehicle-create
description: Create and configure a small controllable vehicle.
metadata:
  capgraph-requires: >-
    chassis-create vehicle-controls
    vehicle-collision third-person-camera
  capgraph-verify-with: "vehicle-verify"
  capgraph-recover-with: "vehicle-collision-repair"
---
```

The exact representation is intentionally simple. It uses the existing metadata extension point rather than introducing new required top-level fields.

In the evaluated implementation, the Agent Skills name is also the graph node identifier. This was sufficient for the experiment but is not presented as a final cross-package identity design. Stable namespace-aware identities and rename compatibility remain open questions.

3.3 External graph index

At startup, Capgraph discovers managed skills, parses frontmatter, validates relation targets, and constructs an in-memory graph view. The complete graph is not exposed to the model.

The model interacts only with a small graph tool surface that supports inspection, expansion, and skill loading. A metadata-only expansion for a supplied root exposes the local relation structure without loading every body. This separates the capability store from the context window.

3.4 Relation semantics

The evaluated relation model contains three edge types.

requires denotes a declared capability dependency. Expansion recursively follows these edges, performs cycle detection, and deduplicates shared dependencies. The resulting execution closure is ordered dependency-first.

verify_with selects the verifier associated with a capability. Verification edges are terminal in the evaluated model; they are not recursively interpreted as execution requirements.

recover_with associates a recovery capability with a root. Recovery is also terminal and, critically, its prose is not batch-loaded during the normal path.

This distinction produces the normal flow:

```
known root
→ expand transitive requires closure
→ load execution closure + verifier
→ execute
→ verify
```

and the failure flow:

```
verify fails with structured evidence
→ load related recovery prose
→ repair
→ verify again
```

No automatic recovery orchestration occurs. The agent still observes the failure and elects to use the available recovery capability.

3.5 Batch disclosure

The initial implementation loaded graph skills progressively, one body at a time. The V0 result showed that this interaction pattern added model turns, tool calls, context replay, and cost even though the graph returned slightly less unique capability context.

Capgraph therefore introduced `load_many` for the known-root closure. A metadata expansion remains separate, but the selected normal execution and verification bodies can be delivered in one structured tool interaction. Recovery remains lazy.

This policy is not merely an implementation convenience; the experiments show that it materially changes the efficiency profile of the graph approach.

4. Experimental Method

4.1 Research question

The formal benchmarks test:

Given a known root capability, does explicit graph structure help the same coding agent complete and verify a workflow more reliably or with less exploration and context than flat skill discovery?

The experiments intentionally do **not** test natural-language root resolution.

4.2 Agent and environment

All formal benchmarks use the same coding-agent harness, `openai-codex/gpt-5.6-luna` at max reasoning, fresh in-memory sessions, and controlled UPBGE editor state. UPBGE 5.3.0 Alpha is used as the executable test domain.

UPBGE was selected because the tasks create concrete editor state that can be independently inspected: objects, physics settings, collision configuration, input properties, camera state, scene linkage, and other properties can be checked after the model session ends.

The agent-facing execution interface exposes fixed UPBGE operations rather than arbitrary Python. Tool definitions are held equal between conditions, and the benchmark design includes explicit controls to prevent tool descriptions from leaking the graph recipe, verifier choice, or recovery relation.

4.3 Flat and Graph conditions

In the **Flat** condition, skill names and descriptions are exposed through normal skill discovery. Full `SKILL.md` files are available through controlled reads. Generated Flat fixtures preserve the same Markdown bodies as Graph skills while removing Capgraph metadata.

In the **Graph** condition, graph-managed skills are excluded from normal flat discovery. The model receives the supplied root, expands graph metadata, and loads the selected closure through the graph interface. Direct reading of Graph-managed source files is blocked.

Both conditions use identical skill prose, task prompts, execution primitives, verifier logic, model settings, and scene state. The intended experimental difference is capability-selection and composition support.

4.4 Independent verification

Model-reported success is not sufficient. After each session, the harness invokes an independent verifier against actual editor state. A run is successful only when this external check passes.

Expected injected verifier failures in Recovery tasks are recorded separately from benchmark failure. This separation prevents a designed runtime fault from being misclassified as tool or task failure.

4.5 Metrics

The benchmarks record:

- independently verified task success;
- benchmark and composition protocol conformance;
- agent turns and tool calls;
- failed tool calls;
- wall-clock duration;
- provider-reported input, output, cache-read, and cache-write tokens;
- provider-reported cost;
- capability catalog bytes;
- graph metadata bytes;
- loaded skill payload bytes;
- selected skill bodies;
- irrelevant skill-body loads;
- verifier selection;
- recovery behavior;
- operation sequence.

Context-byte measurements are kept separate from provider-reported tokens because serialization and frontmatter differ between conditions.

5. Experiments

5.1 V0: shallow workflow and progressive graph loading

V0 uses a five-skill physics-object workflow. The task asks the agent to create a physics-enabled cube with a known root capability, then verify it. Five Flat and five Graph runs are executed.

Both conditions achieve 5/5 independently verified success. Graph reduces unique measured capability context by 4.0%, but Progressive Graph requires more turns, tool calls, duration, provider usage, and cost. Both variants ultimately load all five skill bodies.

Metric	Flat	Graph Progressive
Verified success	5/5	5/5
Median turns	8	12
Median tool calls	10	11
Median duration	25.286 s	28.709 s
Median cost	\$0.004673	\$0.005465
Capability context	21,188 B	20,338 B

V0 therefore provides a useful negative result: a graph does not automatically improve a small workflow, and one-skill-at-a-time disclosure can turn graph structure into extra indirection.

5.2 V0.1: loading-policy ablation

V0.1 isolates the loading policy. Eighteen formal runs compare six Flat, six Graph Progressive, and six Graph Batch sessions. The root skill prose is also revised so that the workflow sequence is not duplicated in the root body; Flat and Graph Markdown bodies remain content-identical.

All three conditions succeed 6/6.

Metric	Flat	Graph Progressive	Graph Batch
Median turns	7.5	8	7
Median tool calls	9.5	10	6
Median duration	21.433 s	21.851 s	18.029 s
Median input tokens	9,414.5	14,973.5	13,173.5
Median cost	\$0.003231	\$0.004269	\$0.003523
Skill bodies loaded	5	5	5

Relative to Graph Progressive, Batch reduces graph load calls from five to one, median tool calls by 40.0%, duration by 17.5%, input tokens by 12.0%, and cost by 17.5%.

Graph Batch still does not uniformly beat Flat: Flat uses fewer median input tokens and remains slightly cheaper in this shallow workflow. The important result is that a substantial part of V0's graph penalty came from the interaction policy rather than from graph traversal itself.

Graph Batch is therefore frozen as the graph loading policy for V1.

5.3 V1: larger composition and controlled recovery

V1 introduces a 24-capability catalog. The normal graph closure contains 14 bodies including the verifier; a fifteenth recovery body becomes relevant only after a controlled failure.

The root is `vehicle-create`. Its transitive structure includes chassis, physics, controls, collision configuration, and camera setup. `object-resolve` is a shared transitive dependency reached from multiple branches but deduplicated to a single body. Nine plausible unrelated capabilities remain outside the root closure, including alternative verifier skills.

Two task variants are used:

1. **Normal:** create and verify a controllable vehicle.
2. **Recovery:** perform the same task, but inject a deterministic one-shot collision-mask defect immediately before the first complete agent-facing verification.

The verifier returns structured expected and actual mask values. The recovery capability repairs only that mask. Independent verification occurs afterward with fault injection disabled.

Four Flat and four Graph formal runs are executed, balanced across Normal and Recovery variants.

6. Results

6.1 Reliability

All eight V1 runs succeed under independent verification:

Metric	Flat	Graph
Independently verified success	4/4	4/4
Benchmark protocol conformance	4/4	4/4
Composition-conformant runs	3/4	4/4
Execution-behavior clean runs	4/4	4/4
Failed tool calls	0	0

Capgraph therefore does **not** demonstrate a reliability advantage. The language model is capable enough to solve every tested Flat task as well.

6.2 Composition and irrelevant skill exposure

Graph loads no irrelevant skill body in any V1 run. Flat loads no irrelevant body in three runs, but one Recovery run reads two plausible alternative verifiers, `static-scene-verify` and `vehicle-static-verify`, before continuing on the correct path.

All eight runs nevertheless select the correct vehicle verifier first.

This is the clearest direct evidence for the narrow composition hypothesis: graph structure constrains the available closure deterministically and prevents irrelevant branch exploration in the observed runs, but the observed Flat exploration does not translate into task failure.

6.3 Capability context

Median measured capability context is:

Context category	Flat	Graph
Always-present catalog	7,954 B	0 B
Graph expansion metadata	0 B	2,768 B
Loaded skill payload	5,970 B	5,087 B
Total	13,924 B	7,855 B

The tested Graph harness therefore exposes 43.6% less median measured capability context.

This should not be interpreted as “graph edges compress context by 43.6%.” The result includes the absence of the Flat always-present catalog and differences in frontmatter and serialization. It is a harness-level context measurement.

6.4 Interaction and provider usage

V1 aggregate medians are:

Metric	Flat	Graph	Graph difference
Agent turns	13	12	-7.7%
Agent tool calls	22.5	11	-51.1%
Duration	41.800 s	38.422 s	-8.1%
Provider input tokens	17,591	17,761	+1.0%
Provider output tokens	1,433	898	-37.3%
Cache-read tokens	48,384	25,344	-47.6%
Median cost	\$0.006181	\$0.005142	-16.8%

The tool-call result requires caution. Graph Batch returns a selected closure in one load interaction, whereas Flat reads selected bodies individually. Consequently, the 51.1% tool-call reduction is partly a mechanical API effect rather than independent evidence of better reasoning.

Provider usage is also mixed at the median level: Graph reduces output, cache reads, and cost, while median input tokens are 1.0% higher.

6.5 Recovery

All four Recovery runs—two Flat and two Graph—observe the same injected collision-mask failure, apply the correct repair, pass a second agent-facing verification, and pass independent verification.

Graph uses its `recover_with` relation to withhold recovery prose until the failure occurs. Flat also finds the correct recovery skill, but does so through normal catalog exploration and preloads the recovery body.

Thus, `recover_with` changes selection timing and context exposure in the observed sample, not recovery reliability.

6.6 Shared dependency handling

Graph deterministically reaches `object-resolve` through multiple incoming dependency branches but loads it once.

Flat loads `object-resolve` in three of four runs and never duplicates it. Therefore, structural deduplication provides a guarantee but produces no observed success or body-count advantage in this small sample.

7. Discussion

7.1 Structured composition can help without improving success

The strongest conclusion from V1 is not that Graph solves tasks Flat cannot solve. It does not. Rather, graph structure narrows the composition process.

With a 24-capability catalog and a smaller relevant closure, Graph follows the declared path consistently, avoids unrelated verifier prose, and delays recovery context until runtime evidence makes it relevant. Flat remains fully capable, but its skill selection is model-driven and therefore somewhat more variable.

This distinction matters for systems engineering. Reliability may already be saturated on a small task with a strong model. In that regime, composition determinism, context exposure, auditability, and behavior variance can still be meaningful properties even when final success is unchanged.

7.2 Loading policy is part of the method

V0 and V0.1 show that graph structure cannot be evaluated independently of how graph-selected context reaches the model.

One-body progressive disclosure looked attractive conceptually because it appeared maximally context-efficient. In practice, the repeated model/tool interactions increased turns, replay, and cost while all five bodies were ultimately needed.

Batch disclosure reversed much of that penalty.

This suggests a general systems lesson: **context selection and context transport are separate design variables**. A better selector can still perform poorly if the delivery mechanism causes excessive interaction overhead.

7.3 Capability dependency is not necessarily context dependency

V1 reveals a counterpoint to deterministic closure loading. Graph Batch always loads the complete declared normal closure. Flat occasionally omits low-level bodies—including `mesh-object-create` in one Normal run and both `mesh-object-create` and `object-resolve` in one Recovery run—without harming independent success.

This motivates an unresolved distinction:

A capability can be required by the operational dependency graph without its full procedural document necessarily being required in model context.

The current Capgraph implementation uses requires both to represent capability structure and to decide which prose enters a batch. The experiments are insufficient to justify a new edge type or loading heuristic, but they show that these two semantics should not automatically be assumed identical.

A future study could distinguish **execution dependency** from **context dependency**, or allow evidence-driven omission of procedural bodies while retaining structural dependency information.

7.4 Flat selection has a genuine adaptive advantage

A flat catalog should not be treated merely as an inferior graph that lacks edges. Its lack of hard structural closure gives the model freedom to skip details it already understands.

In the V1 sample, that flexibility sometimes saves relevant low-level prose. The tradeoff is that the same adaptive search can also explore irrelevant alternatives.

The observed contrast is therefore:

- **Graph:** deterministic complete declared closure.
- **Flat:** adaptive model-selected subset.

Neither behavior is universally superior based on the current evidence.

7.5 Batch loading is a benchmark confound as well as a design result

Graph Batch is the best graph policy found in V0.1, but it also creates an asymmetry in V1: Graph can receive its selected closure through one structured load, whereas Flat loads its self-selected bodies through individual reads.

This means interaction metrics cannot be attributed purely to explicit graph relations.

A useful future ablation would add **Flat Batch**: Flat would retain normal catalog-based selection and no graph information, but could request multiple self-selected skill bodies in one operation. Comparing Flat, Flat Batch, and Graph Batch would better separate composition quality from transport efficiency.

This ablation is not part of the current results.

7.6 Known-root composition is intentionally not retrieval

Capgraph avoids the natural-language-to-capability problem by supplying the root explicitly. This is a major limitation but also an experimental choice.

Prior work such as GoS, SkillDAG, SkillTrace, and AgentSkillOS studies retrieval and structured orchestration together [4, 5, 8, 10]. Their results show that finding the right structural neighborhood is itself a substantial problem.

Capgraph asks what happens **after** that entry point is known. It should therefore be understood as complementary to retrieval systems, not as a replacement for them.

7.7 Interoperability and metadata

Capgraph's use of Agent Skills metadata is intentionally conservative. The standard already permits client-specific string properties [1], so relation encoding does not require changing the required skill format.

This does not imply that Capgraph's metadata keys should become a standard. The Agent Skills ecosystem is actively discussing dependencies, package manifests, relation fields, identities, namespaces, and invocation semantics [2, 3]. Capgraph provides one tested runtime design point within that wider design space.

8. Threats to Validity and Limitations

The experiments have several important limitations.

Small sample sizes. V0 contains 10 formal runs, V0.1 contains 18, and V1 contains 8. The reported differences should not be interpreted as statistically established general effects.

Single model configuration. All formal experiments use `openai-codex/gpt-5.6-luna` at max reasoning. The balance between adaptive Flat selection and deterministic Graph closure may differ substantially for weaker or stronger models.

Single primary domain. UPBGE provides convenient independent state verification, but the experiment does not establish transfer to conventional software repositories, web agents, DevOps agents, or other capability environments.

Known roots. Every formal prompt supplies the root capability. The experiments do not evaluate semantic retrieval, intent resolution, aliases, embeddings, routing, or large-scale graph entry-point discovery.

Loading asymmetry. Graph Batch loads an expanded closure in one operation, while Flat reads its selected skills individually. Interaction metrics therefore combine composition and transport effects.

Declared closure may overexpose prose. Graph loads low-level bodies that Flat can sometimes omit safely.

Tool visibility. Primitive execution operation names remain visible equally to both conditions. The benchmark controls descriptions and parameters to avoid encoding the workflow recipe, but tool affordances can still provide environmental clues.

Identity model. The evaluated implementation uses the Agent Skills name as graph identity. Re-name stability, cross-package identities, and namespaces remain unresolved.

Experimental runtime. UPBGE support is experimentally verified against the tested 5.3.0 Alpha build. Only editor-time behavior is tested; runtime bge control is outside scope.

Security. The agent-facing tool restricts execution to fixed operations, but the underlying local Blender/UPBGE bridge can execute arbitrary Python and has no authentication. Capgraph does not solve that bridge-level security problem.

Context-byte measurement. Byte counts describe returned payloads, not exact provider token attribution. Flat and Graph serialization differ.

9. Reproducibility

The Capgraph repository preserves the benchmark progression rather than replacing unfavorable early results.

For each formal stage, the repository includes a frozen benchmark specification, raw JSONL records, a benchmark report, and provenance hashes. V0, V0.1, and V1 are treated as separate historical experiments.

The formal progression contains 36 model runs:

- V0: 5 Flat + 5 Graph Progressive;
- V0.1: 6 Flat + 6 Graph Progressive + 6 Graph Batch;
- V1: 4 Flat + 4 Graph, balanced across Normal and Recovery variants.

Raw measurements and interpretive analysis are separated so that numeric results can be regenerated from formal records without requiring acceptance of the paper’s interpretation.

10. Conclusion

Capgraph evaluates a deliberately small question in the rapidly expanding space of agent skill graphs: **what do explicit capability relations contribute when the relevant root is already known?**

The answer from the current experiments is nuanced.

A graph is not automatically beneficial. On a shallow five-skill task, progressive graph loading adds overhead and provides no reliability advantage. Loading policy matters enough to change the result: a batch-loading ablation removes most of that interaction penalty. When the catalog expands to 24 capabilities with transitive and shared dependencies, plausible alternatives, verification, and controlled recovery, explicit relations make composition more deterministic and reduce irrelevant capability exposure in the tested harness. They still do not improve final task success.

Across every formal benchmark run, Flat remains capable of completing the task. Its flexibility sometimes lets it omit low-level prose that the deterministic graph loads. This produces a useful tension rather than a simple winner: structured closure offers predictability and relevance guarantees, while adaptive flat selection can exploit model knowledge to skip context.

The central empirical contribution is therefore not that “skill graphs are better.” It is that **capability relations, loading policy, and context disclosure interact in measurable ways**, and these mechanisms should be separated when designing and evaluating agent skill systems.

The next useful evaluation is not a larger Capgraph architecture. It is replication: more genuine tasks, more runs, and at least one additional model while retaining the known-root boundary. A Flat Batch ablation would further isolate graph-based composition from batch transport. Only after those mechanisms are better understood should intent routing, learned relations, or evolving graphs be added.

AI Assistance Disclosure

GPT-5.6 Sol (OpenAI) was used as an AI assistant during the development and documentation of this work, including research discussion, experimental-design critique, related-work research, technical review, analysis review, and drafting assistance. The author, Tino Wening, designed and implemented the project, executed the experiments, made the project decisions, and remains responsible for the methodology, software, benchmark artifacts, interpretations, claims, and final text.

References

- [1] Agent Skills. **Agent Skills Specification**. Open Agent Skills specification, 2026.

- [2] prasadt3. **Proposal: Skill Relationship Fields — adding prerequisite-skills and related-skills to the SKILL.md spec.** Agent Skills GitHub Issue #90, January 2026.
- [3] erdemtuna. **Proposal: Skill Package Manifest for Dependency Resolution and Distribution for Agent Skills.** Agent Skills GitHub Discussion #210, March 2026.
- [4] H. Li, C. Mu, J. Chen, S. Ren, Z. Cui, Y. Zhang, L. Bai, and S. Hu. **Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale.** arXiv:2603.02176, 2026.
- [5] D. Li, Z. Li, H. Du, X. Wu, S. Gui, Y. Kuang, and L. Sun. **Graph of Skills: Dependency-Aware Structural Retrieval for Massive Agent Skills.** arXiv:2604.05333, 2026.
- [6] T. Xia, L. Hu, Y. Sun, M. Xu, L. Xu, S. Wang, W. Xu, and J. Jiang. **GraSP: Graph-Structured Skill Compositions for LLM Agents.** arXiv:2604.17870, 2026.
- [7] X. Li, M. Li, K. Bao, Y. Ma, W. Wang, D. Liu, and F. Feng. **SkillGraph: Skill-Augmented Reinforcement Learning for Agents via Evolving Skill Graphs.** arXiv:2605.12039, 2026.
- [8] T. Bai, Z. Wan, P. Zhou, X. Yu, W. Zhao, Y. You, and I. W. Tsang. **SkillDAG: Self-Evolving Typed Skill Graphs for LLM Skill Selection at Scale.** arXiv:2606.03056, 2026.
- [9] Y. Hao, J. Cai, Q. Zhang, Y. Li, Z. Zhang, C. Shi, and C. Yang. **HiSkill: Empowering LLM Agents with Hierarchical Skill Graphs.** arXiv:2607.25853, 2026.
- [10] Y. Yao, S. Wang, X. Chen, M. Zhang, J. He, B. Luo, and T. Gedeon. **SkillTrace: Traversing a Query-Skill Graph for Composable LLM Agents.** arXiv:2608.02356, 2026.
- [11] TheneoAI. **Skill Writer.** Open-source Agent Skills authoring and graph tooling repository, 2026.
- [12] C. Yang, J. Tian, Z. Wang, X. Liu, M. Ye, and J. Chen. **Learning Globally Reusable Skills for Coding Agents.** arXiv:2608.06153, 2026.
- [13] D. B. Hong, A. Imani, and I. Ahmed. **From Anatomy to Smells: An Empirical Study of SKILL.md in Agent Skills.** arXiv:2607.01456, 2026.